

CS-728 Assignment 1

Deevyanshu Malu (22B1029) Deeptanshu Malu (22B1256)
Dhvanil Gheewala (22B0923) Tanmay Mandaliya (22B1037)

February 2026

1 Task 1: Glove pre-training

1.1 Chosen Hyperparameters

To balance model performance with computational efficiency, we established a fixed set of hyperparameters for the final embeddings. To avoid an explosion of experimental runs, we adopted a strategy of fixing the embedding dimension first and prioritizing the optimization of remaining parameters.

The final hyperparameters chosen for this implementation are as follows:

- Context Window ($w=3$): This parameter dictates the scope of the context, considering 3 words to the left and 3 words to the right of the target word.
- Optimization: The gradient descent process was executed with a learning rate of 0.1 over 75 epochs.
- Weighting Hyperparameters: We utilized the standard weighting function parameters found in the literature, specifically $x_{max}=100$ and $\alpha=0.75$.

1.2 Plots

Loss Plots

The following figures illustrate the training loss over the epochs. These results were obtained using the selected hyperparameter configuration of a context window size of 3 and a learning rate of 0.1 over 75 epochs.

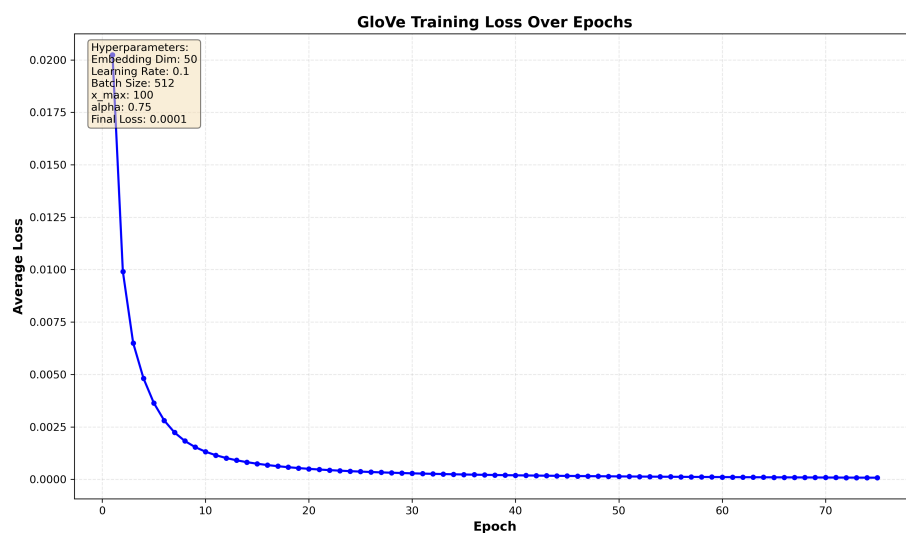


Figure 1: $\text{dim} = 50$, $w = 3$, $\text{lr} = 0.1$

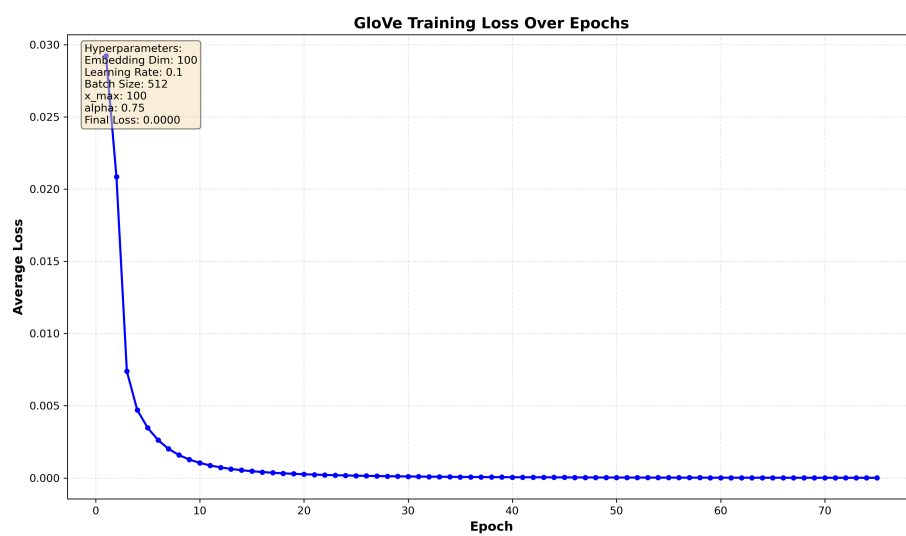


Figure 2: $\text{dim} = 100$, $w = 3$, $\text{lr} = 0.1$

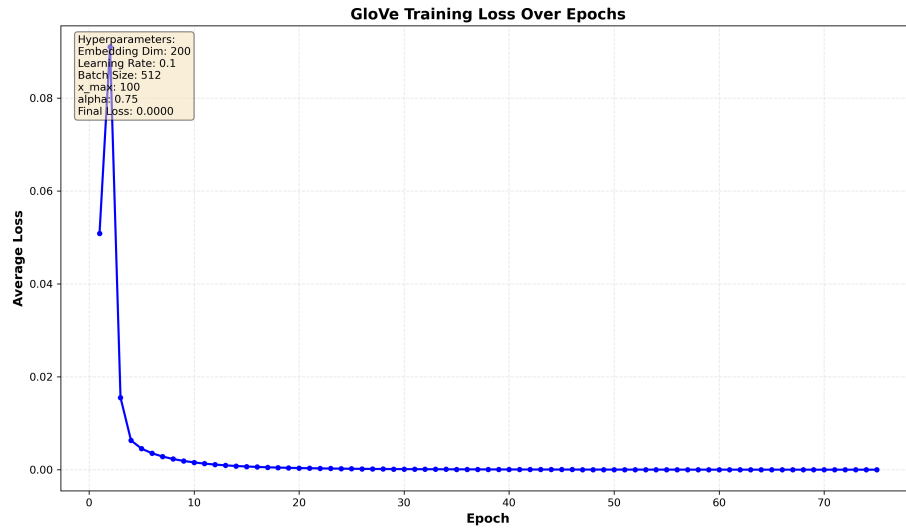


Figure 3: $\text{dim} = 200$, $w = 3$, $\text{lr} = 0.1$

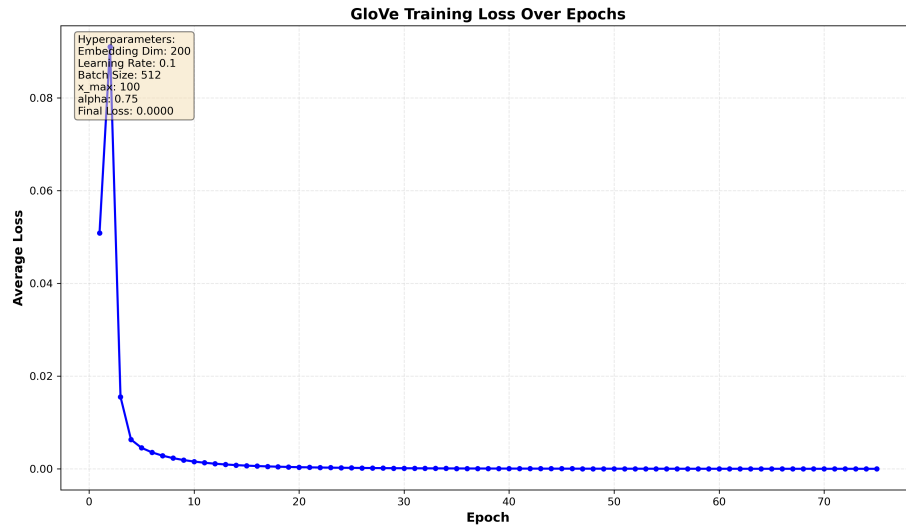


Figure 4: $\text{dim} = 300$, $w = 3$, $\text{lr} = 0.1$

Latency Plots

The following figure illustrates the embedding dimension vs training time per epoch. This result was obtained using the selected hyperparameter configuration of a context window size of 3 and a learning rate of 0.1 over 75 epochs

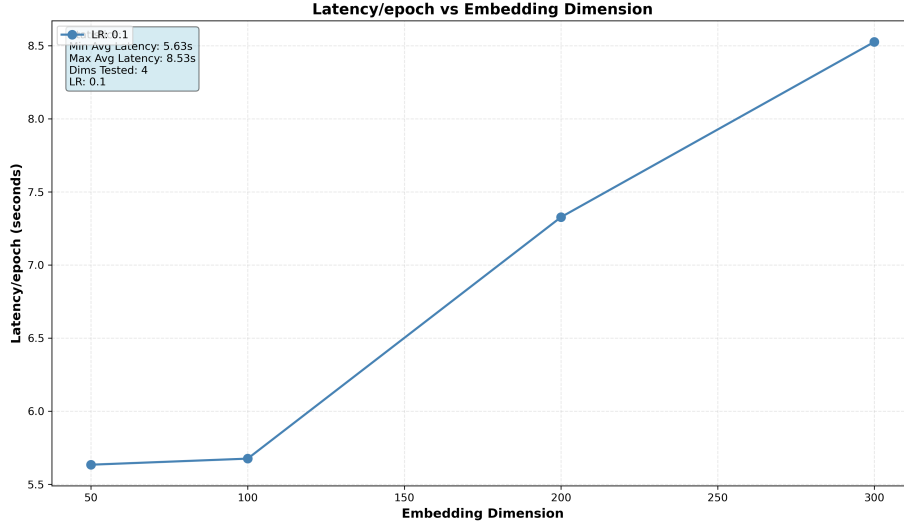


Figure 5: embedding dimension vs training time/epoch

We selected a final embedding dimension of $d=100$. While the training was comparable to that of $d=50$, the 100-dimensional embeddings demonstrated better convergence (lower loss) and improved correlation metrics.

1.3 5-nearest neighbours

The five nearest neighbours for the following words along with their cosine similarity are:

- President

dimension	1-nearest	2-nearest	3-nearest	4-nearest	5-nearest
50	arrival	SKIING	reaches	Rich	re-nomination
100	Gordon	Winner	exploit	competing	'7.40'
200	Sepp	ceased	Silicon	Corsica	believe
300	enterprise	BEIJING	spectacular	Berti	SBC

- city

dimension	1-nearest	2-nearest	3-nearest	4-nearest	5-nearest
50	alleged	'688,000'	detainees	Craiova	Warszawa
100	Yigang	Professional	Tomasson	Krishnamoorthy	Bunol
200	Gulf	'841'	famished	Newcastle	CAPS
300	Kilmarnock	ARA	Dijk	Army	Guiana

- computer

dimension	1-nearest	2-nearest	3-nearest	4-nearest	5-nearest
50	Miyagi	'48.4'	finding	supplies	Carretero
100	Fulgencio	Frazier	Longyearbyen	secs	hitting
200	social	coaching	Davids	erased	'16.41'
300	Perfetti	play	tablets	Ishwari	Schalkwyk

2 Task 2: SVD pipeline

For this task, we construct a term-document matrix Z of size $N \times D$ where N is the vocabulary size and D is the number of unique documents. Each entry Z_{ij} represents the number of times word i appears in document j . The passages are tokenized using a simple whitespace-based split, and the word counts are accumulated per document.

We use `scipy.sparse.lil_matrix` to build the matrix (since it is very sparse), and convert it to CSR format before applying the SVD. The matrix has shape 25013×64172 with 224,040 non-zero entries.

The SVD decomposes Z into $U\Sigma V^T$, and we truncate it to rank d . The final token representations are obtained as $W_d = U_d \Sigma_d$, which is directly returned by `sklearn.decomposition.TruncatedSVD.fit_transform`. We generate embeddings for $d \in \{50, 100, 200, 300\}$.

2.1 Explained Variance

Table 1 shows the cumulative explained variance ratio for each embedding dimension. Higher d captures more of the original matrix’s information, but with diminishing returns.

d	Explained Variance
50	0.3847
100	0.4269
200	0.4824
300	0.5202

Table 1: Cumulative explained variance ratio for each SVD dimension

2.2 5-nearest neighbours

We report the top-5 nearest neighbours (by cosine similarity) for the same three words used in Task 1. The results are shown in Table 2.

Word	d	Top-5 Nearest Neighbours
president	50	condemns, dismay, Critics, extremist, fringe
	100	condemns, dismay, code, Manuel, fringe
	200	condemns, dismay, code, NBC, Manuel
	300	condemns, dismay, white, violence, NBC
city	50	staying, beautiful, Most, tons, stunning
	100	Santa, Festival, Make, stunning, century
	200	Festival, spend, stunning, tons, century
	300	Festival, spend, century, town, celebrations
computer	50	computers, priced, speaker, shoe, steel
	100	speaker, priced, steel, revision, surprisingly
	200	thin, speaker, void, bands, collections
	300	Apple, void, Stainless, foreshadowed, bands

Table 2: Top-5 nearest neighbours for SVD embeddings across dimensions

At lower dimensions ($d=50$), the SVD embeddings tend to cluster many loosely related words together since the representation space is limited. As the dimensionality increases, the neighbours become more semantically relevant – for instance, “computer” at $d=300$ retrieves “Apple” as its closest neighbour, and “city” retrieves “town” and “celebrations”. This suggests that higher-dimensional SVD embeddings capture finer semantic distinctions.

3 Task 3: NER using CRF

3.1 Features

The CRF features for each word are:-

- Bias=1
- Its prefixes of length 1, 2, and 3
- Its suffixes of length 1, 2, and 3
- Boolean indicating the word is at BOS
- Boolean indicating the word is at EOS
- Its lowercase form
- Boolean indicating if all the characters are uppercase letters
- Boolean indicating if the first character is an uppercase letter
- Boolean indicating if all the characters are digits

We also add the features below for the previous and next word:-

- Their lowercase form
- Boolean indicating if all the characters are uppercase letters
- Boolean indicating if the first character is an uppercase letter
- Boolean indicating if all the characters are digits

3.2 Most Important Features

The most important features are those involving the **previous and next words**. These features help the CRF model contextualize and, hence, give better results. The metrics with and without these features are shown in Table 3.

3.3 Results

Training Type	Accuracy	Macro-F1
With context	95.8	81.1
Without context	95.1	78.5

Table 3: Token-level and Macro-F1 Accuracy using CRF

4 Task 4: NER using feature learning

For this task, we normalise the embeddings from the previous tasks to improve stability during training. Further, we keep the embedding trainable to further improve the NER model’s performance.

To handle out-of-vocabulary (OOV) words, we take the average of all the word embeddings and assign that as the embedding to a common **(UNK)** token. This is a good initialisation method, and during training, it is refined to provide a better estimate.

4.1 Model Architecture

Our model is a simple feed-forward neural network. Each input token is first mapped to a dense vector representation using an embedding layer, which can be initialised with pretrained embeddings (from GloVe or SVD). These embeddings are then passed through a 2-layer MLP: a linear transformation to a 128-dimensional hidden representation, followed by a ReLU activation, and a final linear layer that projects the hidden representation to a 9-dimensional output label space (since there are 9 labels). The model produces logits for each token independently, without explicit contextualization between neighbouring tokens.

4.2 Results

From the results in Table 4, we can see that the best training configuration is with GloVe and d=200 or 300.

4.3 MLP vs CRF

Compared to a CRF, the proposed MLP-based NER model is expected to **perform worse** as is evident from the results in Table 3 and 4. This is because the

Embedding	d	Accuracy	Macro-F1
GloVe	50	89.3	58.8
	100	89.8	60.2
	200	90.6	62.2
	300	90.8	62.1
SVD	50	90.1	61.2
	100	90.4	61.6
	200	90.6	61.4
	300	90.8	61.5

Table 4: Token-level Accuracy and Macro-F1 Score using MLP

MLP model predicts labels for each token independently and does not account for the dependencies between neighbouring labels. In contrast, a CRF explicitly models transitions between sequential labels and enforces valid tagging structures, such as those defined by the BIO scheme. This capability is significant for NER tasks. By considering the global sequence during decoding, CRFs typically produce more consistent and accurate entity predictions, resulting in higher token accuracy and F1 scores than token-wise classification models like the MLP.

5 Task 5: SVD Performance Boost

For this task, we have chosen $d=300$.

5.1 5-nearest neighbours

We apply a TF-IDF transformation to the term-document matrix before running the SVD. The TF-IDF weighting downweights terms that appear across many documents and emphasizes terms that are more discriminative. We pick 5 diverse words and compare the top-5 nearest neighbours from raw SVD and TF-IDF SVD at $d=300$. The results are shown in Table 5.

Word	Raw SVD	TF-IDF SVD
president	condemns, dismay, white, violence, NBC	named, shared, shows, using, school
running	liberated, compassion, workout, Run, lets	entire, men, Richard, search, passing
London	despite, challenging, reorganization, transmitted, content	designed, star, surgery, break, room
beautiful	Rodrigo, famous, historic, park, island	cool, realize, widely, Do, River
attack	nightmare, 38th, unleashed, Seoul, missiles	lived, shot, go, strong, City

Table 5: Top-5 nearest neighbours: Raw SVD vs TF-IDF SVD at $d=300$

The raw SVD neighbours tend to be more topically related (e.g., “attack” retrieves “missiles”, “Seoul”), while the TF-IDF SVD neighbours are more generic.

This is because the TF-IDF weighting spreads out the similarity scores more evenly, reducing the dominance of high-frequency co-occurrence patterns.

5.2 Results

Embedding	d	Accuracy	Macro-F1
SVD	300	90.8	61.5
SVD with TF-IDF		90.6	61.6

Table 6: Token-level Accuracy and Macro-F1 Score using MLP

From Table 6, we can see that the TF-IDF embeddings perform very similarly compared to the regular embeddings. This may be due to having a very similar initialisation and keeping the embeddings trainable. As a result, the models are trained similarly and produce similar embeddings.

6 GenAI Usage

<https://chatgpt.com/share/699071cf-6420-8001-81dc-f31117b2d9cf>